SECURITY IMPLEMENTATION CORRECTION

This page supersedes older statements in the original report that describe the listed security controls as unresolved, pending, or proposed work. Those statements reflected the project before the later implementation work.

SECURITY CONTROLS NOW IMPLEMENTED

- JWT authentication, active-account checks, and current-database role checks.
- Administrator, Inspector, and User role authorization on protected routes.
- Inspector station/worker scoping and User-to-worker ownership checks.
- Typed request validation, safe errors, strict MongoDB validators, and indexes.
- Helmet headers, explicit CORS allowlist, request-size limits, and rate limits.
- Single-use, expiring password recovery links with hashed reset tokens.
- Authenticated WebSocket alerts scoped to the assigned inspector.
- Protected AI uploads: inspector authorization, image limits, private service key forwarding, controlled inference concurrency, and safe failures.

VERIFICATION COMPLETED

The backend regression suite passed 27 of 27 tests after these changes. The tests cover authorization, validation, rate limiting, recovery, WebSocket scope, AI upload protection, database validation, and User-role ownership.

REMAINING DEPLOYMENT LIMITS

The live site is a free demonstration: Vercel hosts the frontend while the backend, database, and AI model stay on the group laptop behind a temporary Cloudflare Quick Tunnel. Browser tokens also remain in localStorage. These are deployment/design limits, not missing implementations of the listed controls.

CURRENT SECURITY IMPLEMENTATION STATUS

This table supersedes the older pre-implementation security status table in the original report.

Security measure	Status	Easy-to-understand proof

GROUP 4 • BSIT 2-07

WearAware

Checkpoint 02: Deploying and Securing a MERN Application

ITE 314: ADVANCED DATABASE SYSTEMS

Project Information	Details
Project Title	WearAware: PPE Compliance Monitoring and Worker Portal
Group/Team	Group 4
Members	Ivan Miguel F. Chen Brenan Josh Cervantes John Nathaniel T. Tercero
Course/Section	ITE 314: Advanced Database Systems / BSIT 2-07
Instructor	To be supplied by the group
Date	Prepared: 23 September 2026 Submission date: to be confirmed
Frontend Deployment URL	Not supplied or verified. Local frontend: http://localhost:5173
Backend/API Deployment URL	Not supplied or verified. Local API: http://localhost:5000
Repository URL	https://github.com/voncheems/WearAwareRM

Submission scope

This activity documents the available implementation, observed local behavior, and deployment plan. It does not claim that a public production deployment or all required security controls have been completed. The architecture uses the native MongoDB driver; Mongoose is not installed.

Names and section were taken from the project's About page and should be confirmed before submission. The report reflects the local working tree, including changes that may not yet be on GitHub. No application source code or real database records were changed to prepare this report.

Table of Contents

1. Project Overview

2. MERN Architecture

2.1 Request-Response Flow

3. Project Features and REST API

4. Security Analysis of Our Project

5. Authentication and Authorization

5.1 RBAC Matrix

6. Secure Development Practices

7. Database Design and Security

8. Deployment Plan

8.1 Deployment Architecture

9. Security Testing

10. Deployment Verification Checklist

11. Evidence and Screenshots

Reflection Questions

12. Final Project Information

13. Suggested Evaluation Rubric

Appendix A. Source Evidence Index

Appendix B. Safe Configuration Example

How to read the evidence

Implemented means the control is present in source. PASS (local) means a recorded check passed in an isolated local environment. PARTIAL means the control covers only some paths. GAP means a missing or failing control was identified. PENDING means the result needs a deployed environment or information from the group. A passing regression suite is not proof that every security requirement is satisfied.

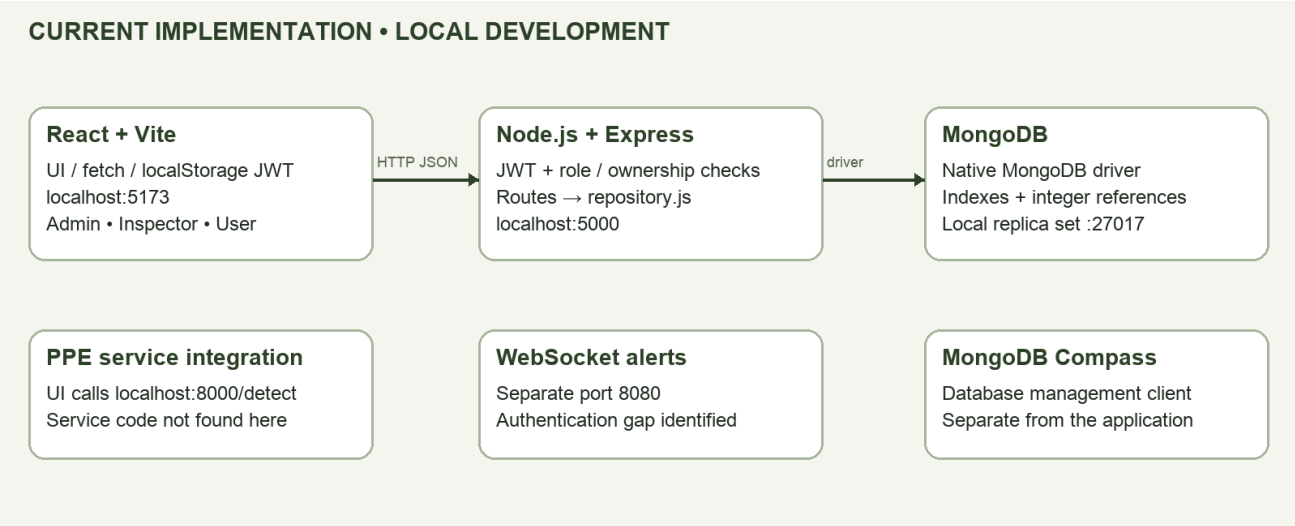
The template’s requested sections are retained. Its Mongoose, production URL, and screenshot prompts are treated as reporting requirements, not evidence that those features already exist. Figures 1–3 are real browser captures; Figures 4–6 and 10 are visual presentations of recorded test/query output. Figures 7–9 remain explicitly pending.

1. Project Overview

Item	Group Response
Project Title	WearAware: PPE Compliance Monitoring and Worker Portal
Problem/Need Addressed	Manual PPE checks and scattered records make it difficult to review violations, monitor station activity, and give workers visibility into their own compliance history.
Target Users	Administrators manage accounts, workers, and stations; inspectors perform checkpoint duties; linked worker accounts view their own records. Visitors can view the public information pages.
Main Purpose	Record PPE inspection outcomes and present compliance information to the appropriate users, supporting workplace safety monitoring and follow-up.
Major Features	Role-based login; worker and station management; QR-based worker identification; PPE detection service integration; violation records and notifications; admin and inspector dashboards; read-only worker profile, history, QR code, and personal compliance rate.

2. MERN Architecture

React communicates with the Express API through fetch requests. Express runs on Node.js and calls repository.js, which uses the native MongoDB driver for queries, aggregation joins, indexes, and transactions. This is a MongoDB–Express–React–Node stack, but it does not implement the template's requested Mongoose ODM layer. If Mongoose is mandatory, this remains an assignment requirement to address. [S1–S5]



Architecture A. Current local implementation and supporting integrations.

Component	Technology Used	Role in Our Project
Frontend	React 19 / Vite 7	Marketing pages, role dashboards, forms, transitions, and API requests.
Backend	Node.js / Express 5	HTTP routes, authentication, role checks, validation, and business logic.
Data-access layer	MongoDB native driver 6; no ODM	Repository functions and application-level reference checks; no Mongoose models.
Database	MongoDB replica set	Persistent collections, indexes, and transactional relationship cleanup.
API communication	REST-style JSON over local HTTP	JWT bearer authorization; HTTPS is a production requirement, not demonstrated here.

2.1 Request-Response Flow

Example: a worker opens “My overview” after signing in. The compliance percentage uses all of that worker’s recorded scans, not only the current history page.

Step	What Happens in Our Application
1. React/UI Event	UserDashboard loads the personal overview and reads the login token from localStorage.
2. HTTP Request	GET /api/user/dashboard?page=1 with Authorization: Bearer <token>.
3. Express Route	/api/user router applies requireAuth and requireRole('user') before its dashboard handler.
4. Controller/Logic	Validate page; reload the authenticated account; resolve worker_id from MongoDB. Client-supplied worker/user IDs do not determine ownership.
5. MongoDB Operation	Find the linked worker; aggregate compliant/violation totals; match detections by worker_id; sort, skip, and limit history to 20; join station labels.
6. API Response	JSON includes account, worker profile, stats, history, page, pageSize, and totalPages. An unlinked account receives HTTP 409.
7. UI Update	Display profile, station, history, and compliance = compliant ÷ total × 100, rounded to one decimal. With no scans, show “No scans yet” rather than 100%.

3. Project Features and REST API

Representative implemented endpoints. “A” = admin, “I” = inspector, “U” = linked worker user. These are API roles, not database accounts. [S3, S6–S9]

Resource	Method	Endpoint	Purpose	Protected?
Authentication	POST	/api/auth/login	Issue an 8-hour JWT.	No
Session	GET	/api/auth/me	Return current account.	Any active role
Accounts	GET, POST	/api/users	List/create accounts.	A
Accounts	PUT, DELETE	/api/users/:id	Edit/delete account.	A
Workers	GET	/api/workers	List workers (broad read).	A, I
Workers	POST	/api/workers	Create worker.	A
Workers	PUT, DELETE	/api/workers/:id	Edit/delete worker.	A
Stations	GET, POST	/api/devices	List / create stations.	GET A,I; POST A
Stations	PUT, DELETE	/api/devices/:id	Edit/delete station.	A
Detections	POST	/api/detections	Record scan outcome.	I
Inspection	GET	/api/inspector/detections	Read own inspection history.	I
Override	PATCH	/api/inspector/detections/:id/override	Mark an owned detection compliant.	I
Admin report	GET	/api/admin/stats	Aggregate compliance.	A
Worker portal	GET	/api/user/dashboard	Own profile, history, compliance.	U
Password reset	POST	/api/auth/forgot-password	Request administrator assistance.	No

The UI also sends camera images to a separate localhost:8000/detect integration. Its service implementation and detection accuracy were not verified in this workspace. MongoDB Compass is a database client and does not replace the Express server. [S10]

4. Security Analysis of Our Project

This review identifies concrete implementation risks. Planned controls below are recommendations for the next revision, not changes completed during document preparation.

Area	Potential Risk in Our Project	Impact	Planned Control
Authentication	JWT lives in localStorage; recovery responses distinguish nonexistent and pending accounts. No login throttle is configured.	Token theft after XSS; account enumeration; repeated guessing.	Add throttling and uniform recovery responses; review session storage and expiration policy.
Authorization / RBAC	Worker portal ownership is enforced, but general worker/device GET routes allow broad inspector access; scan creation lacks a station-ownership check.	Inspectors may access unrelated worker data or submit mismatched scan associations.	Scope all inspector reads and writes to assigned stations and validate worker–station–inspector relationships.
Input validation	Repository rejects object-valued query fields and invalid IDs, but server-side format/length checks are inconsistent and mostly hand-written.	Malformed data, misleading 500 errors, and reliance on browser validation.	Define server-side request schemas, allowed fields, field lengths, and consistent 400 responses.
Database	Local startup binds loopback but does not configure MongoDB authorization or TLS. Production DB permissions are unverified.	An exposed or overprivileged DB could reveal or alter worker records.	Use a dedicated application DB identity, restricted network access, TLS, backup/restore tests, and retention rules.
API	No rate limiter or security-header middleware found. Malformed JSON returned an HTML stack in a local probe.	Resource abuse and disclosure of internal paths/implementation details.	Centralize safe errors; cap requests; configure appropriate response headers; disable framework disclosure.
Passwords	users.password_hash is bcrypt-hashed, but reset handling also stores and returns temp_password in plaintext.	Compromise of reset records can reveal usable passwords.	Replace temporary passwords with expiring single-use reset tokens; remove plaintext reset credentials and invalidate affected secrets.
Deployment	Frontend/API/inference URLs and CORS are localhost-specific. No live HTTPS URLs were provided.	Broken production requests or unencrypted transport if deployed unchanged.	Environment-based endpoints; HTTPS API/frontend/inference; controlled CORS; authenticated WSS.
Other: live alerts	WebSocket server has no connection authentication shown and sends violations to every open client.	Worker names, IDs, and violation details may reach unauthorized recipients.	Authenticate connections, enforce per-inspector subscriptions, and minimize alert payloads.

Priority before public deployment: fix credential recovery, WebSocket access, inspector scope, and safe error handling; then verify TLS, database access, validation, and request limits. The 16 passing regression tests cover selected behaviors and do not remove these findings. [S3–S9, S11]

5. Authentication and Authorization

POST /api/auth/login looks up the submitted email, checks that the account is active, compares the password with bcrypt, and signs an 8-hour JWT. Protected requests verify the signature, reload the account, reject deactivated users, and check whether updated_at makes the token stale. The current database role replaces the token's role claim, so an old claim cannot restore admin access. [S3, S4]

React uses the returned role to select a dashboard. Server middleware remains the enforcement point. A "user" account must link to a real worker, and a unique partial index prevents two accounts from claiming the same worker. The worker portal derives ownership from the current account, not a request parameter. [S5–S7]

Role	Description	Allowed Functions	Restricted Functions
admin	System administrator	Manage users, workers, stations, assignments, activity, reset requests, and organization reports.	Inspector-only scan and override APIs and user-only dashboard are not granted automatically.
inspector	Checkpoint operator	Record detections; review inspector history; handle own alerts; update own profile and assigned-station workflows.	User administration and admin reports. General worker/device reads remain too broad and are documented as a gap.
user	Linked worker account	View own profile, station, scan history, all-time compliance, and employee-ID QR code.	Admin/inspector APIs; record mutation; other workers' history.
Guest	Unauthenticated visitor	Marketing pages, login, contact submission, and recovery request.	Protected database resources and dashboards.

5.1 RBAC Matrix

The matrix describes current API access; "own" and "assigned" indicate intended data scope. Role gates are explicit, so admin does not inherit every inspector/user endpoint.

System Function	Admin	Inspector	User	Guest
View public pages	Yes	Yes	Yes	Yes
View worker/device registry	Yes	Yes (broad)	No	No
Create/edit/delete users, workers, stations	Yes	No	No	No
Manage user activation and worker links	Yes	No	No	No
View organization-wide reports	Yes	No	No	No
Create PPE scan record	No	Yes*	No	No
Review inspector history / override own scan	No	Own	No	No
Assign workers through inspector workflow	No	Assigned station	No	No
View personal worker dashboard / compliance	No	No	Own	No
Download personal employee-ID QR	No	No	Own	No

*Scan creation is inspector-only, but the submitted worker and station relationship still needs stronger ownership validation. A legacy "scanner" string remains in frontend routing; no complete scanner dashboard is rendered, so it is not represented as a fourth operational account role. QR contents identify a worker and are not authentication credentials. [S1, S3, S8, S10]

6. Secure Development Practices

Security Control	Implementation in Our Project	Evidence / Status
Input Validation	Positive integer IDs, page bounds, scalar query fields, typed PPE arrays/booleans, detection/status enums, and valid unique worker links. Full server-side schemas remain incomplete.	PARTIAL — S5–S7; Figure 5; worker portal tests.
Safe Database Queries	MongoDB filters are constructed in repository functions; normalize rejects object-valued query inputs. Updates use fixed operators such as $set. Raw client filter objects are not accepted by these helpers.	IMPLEMENTED in reviewed repository paths — S5; invalid worker_id object rejected by tests.
Password Hashing	Account creation and password changes use bcrypt with cost 10. Reset workflow also keeps plaintext temp_password, so password storage is not fully secure.	PARTIAL / GAP — S3; supplemental bcrypt check; reset source at lines 469–513.
Authentication	Signed JWT, 8-hour lifetime, signature verification, active-account check, and timestamp-based invalidation.	PASS (local) — S3–S4; database and portal tests.
Authorization / RBAC	Explicit allowlists, fresh DB role, account-to-worker ownership, and selected inspector scoping. Broad read/write gaps remain outside the worker portal.	PARTIAL — S4, S6–S9; Figure 4.
Least Privilege	Workers have a read-only portal. Inspector registry reads, WebSocket access, and production DB permissions need improvement.	PARTIAL — S6, S8–S9; no deployed DB evidence.
Secure Error Handling	Many route catches return generic JSON. Malformed JSON reaches an Express HTML error response with a stack and filesystem path in development.	GAP — security-probes.json; central error handler still needed.
Security Headers	No Helmet or equivalent header policy found. Sample response lacked CSP, HSTS, and X-Content-Type-Options and disclosed Express.	GAP — S2–S3; recorded HTTP header probe.
Rate Limiting	No configured request limiter identified for login, recovery, contact, or detection submission.	GAP — source inspection; load/burst test not performed.
Environment Variables	Backend uses dotenv and MONGODB_URI, MONGODB_DB, JWT_SECRET, ports, and mail settings. No secret values are reproduced in this report.	IMPLEMENTED locally — S2–S5. Current secret-file history was not audited.
HTTPS / TLS	Local browser/API communication is HTTP. No public HTTPS certificate or hosted endpoint has been verified.	PENDING production evidence — Figures 7–9.
Database Security	Unique indexes, reference checks, selected delete transactions, and loopback local binding are present. DB authorization, TLS, backup policy, and production least privilege are not verified.	PARTIAL — S5, S11; Figure 6. No database-level schema validators found.

Evidence IDs refer to Appendix A. “Implemented” applies only to the cited paths, not to an independently certified production system.

7. Database Design and Security

MongoDB stores the application records. The project keeps integer business IDs from its earlier database design, while MongoDB also maintains internal _id values. Relationships use integer references and $lookup aggregation joins rather than Mongoose populate. PPE lists are embedded arrays. [S5]

Collection	Purpose / Important Fields	Embedded / Referenced	Security Consideration
roles	Account permissions: id, name.	users.role_id → roles.id.	Unique role names; account management is admin-only.
users	Login/account: id, role_id, worker_id, email, full_name, password_hash, is_active, updated_at.	References roles and optionally one worker.	Unique email and partial unique worker_id; use limited response projections; never return password hashes.
devices	Checkpoint/station: id, device_id, label, location, inspector_id, required_ppe, is_active.	References inspector user; embeds required_ppe array.	Restrict management; scope inspector reads and scan submissions more tightly.
workers	Worker registry: id, employee_id, full_name, position, contact_number, device_id, status.	References station; referenced by accounts and detections.	Unique employee ID; protect identity and contact data; preserve ownership constraints.
detections	Scan result: id, worker_id, device_id, inspector_id, result, detected_ppe, missing_ppe, confidence_score, photo_url, detected_at.	References worker, station, inspector; embeds PPE arrays.	Filter by owner/role; minimize photos and identifiers; define retention and access policy.
notifications	Violation alerts: id, detection_id, inspector_id, is_read, created_at.	References detection and inspector.	HTTP notification routes scope to inspector; separate WebSocket path still needs authorization.
password_reset_requests	Recovery queue: id, email, reason, status, temp_password, created_at.	Email-based association; no repository foreign-key relationship.	Plaintext temp_password is an unresolved gap. Use expiring reset tokens instead.
_counters / _migration	Internal ID allocation and migration state.	Atomic counters; postgres-v1 migration marker.	Internal operational data; no public editing interface; protect migration credentials and backups.

Integrity and operational controls

ensureIndexes creates unique record IDs, unique role/email/device/employee identifiers, and the partial unique worker-account link. Repository checks reject unknown referenced records. Selected deletion operations run in transactions: deleting a worker clears linked account and detection worker IDs while preserving historical scans. Tests confirm that a newly unlinked worker account is blocked with HTTP 409.

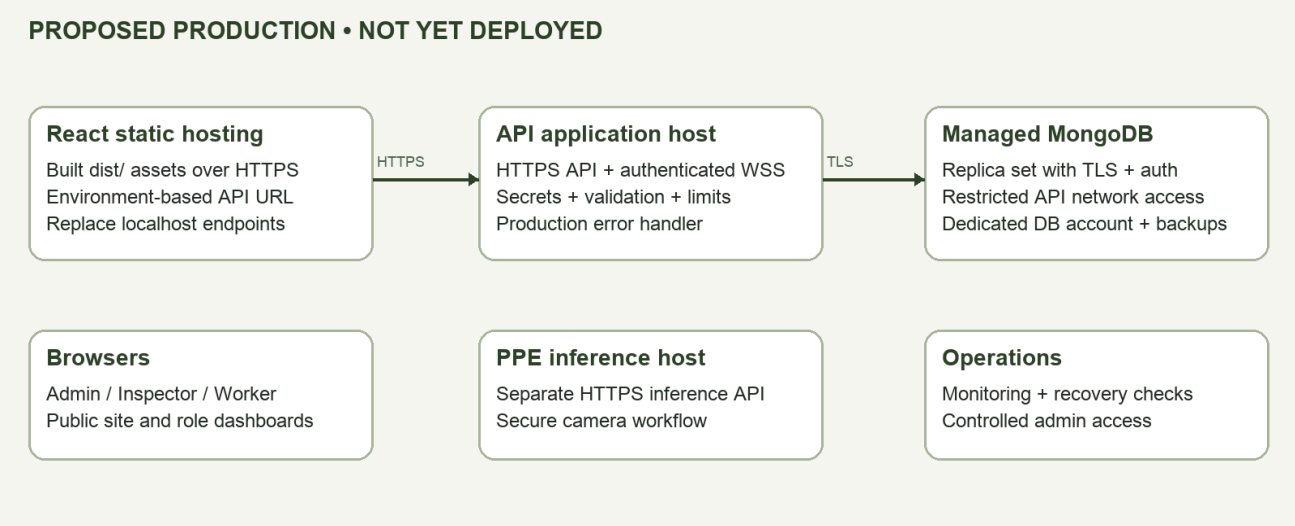
Startup requires a completed postgres-v1 migration marker before it starts the API. A new empty database therefore needs an explicit migration/initialization plan. The local launcher creates a replica set bound to 127.0.0.1; it does not configure authorization or TLS. Production access controls and restore procedures remain pending. Compass is an administrative viewer/editor for this database, not its hosting or application runtime.

8. Deployment Plan

A public deployment was not demonstrated or supplied. The following is the project's planned transition from the reviewed local implementation to production, not a record of completed hosting work.

Component	Local Environment	Production Environment	Status
React Frontend	Vite at localhost:5173; localhost API/inference URLs.	Build dist/ and host over HTTPS; configure public API/inference base URLs.	Local captured; production pending.
Node/Express API	HTTP :5000; WebSocket :8080; localhost CORS.	Persistent Node host/reverse proxy with HTTPS/WSS, fixed allowed origins, and authenticated alerts.	Local tests passed; production pending.
MongoDB	Local replica-set launcher; isolated databases for this report's tests.	Managed replica set (e.g., Atlas) or securely administered equivalent; dedicated application account and restricted access.	Production target/permissions unconfirmed.
Environment Variables	Backend dotenv configuration; frontend URLs still hardcoded.	Host-managed secrets; separate public frontend configuration; no secrets in browser builds.	Backend local support; production pending.
HTTPS	Local frontend and API use HTTP.	Valid certificates for public frontend/API/inference and WSS alerts.	Not verified.
PPE Inference	Frontend calls localhost:8000/detect.	Separately hosted and secured inference service with compatible image request contract.	Integration identified; service implementation not verified.

8.1 Deployment Architecture



Architecture B. Proposed target only; no hosting provider or live URL is claimed.

Deployment sequence and acceptance

First resolve the high-priority security gaps in Section 4. Then provision a restricted MongoDB replica set, migrate/initialize and verify records plus indexes, configure API secrets and allowed origins, deploy the API and inference service, and build the frontend against their HTTPS URLs. Finally verify all three roles, CRUD, owned history, camera access, WSS delivery, backups/restoration, safe errors, and throttling against the deployed environment. Repeat the Section 9 checks and replace pending evidence with actual live captures.

9. Security Testing

Executed 23 September 2026. Existing tests: npm --prefix src/wearawarebackend test → 16 passed, 0 failed. Tests and supplemental probes used disposable MongoDB replica sets and the existing Express app, without writing to the configured application database. No production penetration or load testing was performed.

Test Case	Procedure	Expected Result	Actual Result	Status
Invalid Login	Submit wrong password for a synthetic account.	401; generic credential error.	401: “Invalid email or password.”	PASS local
Unauthorized Route	Worker requests GET /api/users.	No admin records; 403.	403: “Access denied.”	PASS local
Role Restriction	Worker calls admin/inspector and detection write APIs; test stale admin claim.	403 using current DB role.	Assertions passed in user-portal.test.js.	PASS local
Invalid Input	Submit page=-1; test missing, unknown, object-valued, and duplicate worker links.	400 for bad input; 409 for duplicate link.	Expected 400/409 returned; unique index enforced.	PASS local
Protected API Without Token	GET /api/user/dashboard without bearer token.	401; no worker data.	401: “No token provided.”	PASS local
Password Storage Check	Compare synthetic account hash using bcrypt; review recovery persistence.	Password hash only; no recoverable plaintext.	bcrypt check passed; reset source stores plaintext temp_password.	PARTIAL / GAP
Secure Error Response	POST malformed JSON “{” to login.	Generic 400 JSON with no stack/path.	400 HTML; stack and local path present in development.	FAIL local
HTTPS Check	Verify public frontend/API URLs and certificate.	Trusted HTTPS for all public services.	No deployed URLs supplied; local tests used HTTP.	PENDING
Rate Limit Test	Review limiter; plan bounded repeated login attempts on owned deployment.	Defined threshold returns 429 safely.	No limiter found; burst test not performed.	GAP / not run
Database Access Check	Review local launcher; plan authorized/unauthorized DB login and network checks.	Only permitted DB identity and network can connect.	Loopback binding found; auth/TLS not configured by launcher; deployed access not tested.	PARTIAL / pending

Additional checks and limits

Own dashboard returned HTTP 200 and 66.7% compliance for 4 compliant scans out of 6 synthetic scans. Existing tests also prove pagination does not change the all-history percentage, forged worker IDs do not change ownership, no-scan compliance is null, and deleted worker links fail closed. The sampled API response lacked CSP, HSTS, and X-Content-Type-Options headers and exposed X-Powered-By: Express.

The regression suite covers the cases it asserts; it is not an exhaustive audit. The malformed-JSON result was observed in local development, so production behavior still needs verification after a central error handler and production configuration are added. Raw records: evidence/backend-tests.txt and evidence/security-probes.json.

10. Deployment Verification Checklist

This is a truthful readiness checklist, not a claim of completed production deployment. “Local only” items must be repeated on the eventual hosted application.

Requirement	Verification Status
Frontend accessible through deployed URL	PENDING — local homepage/login captured; no live URL.
Backend/API accessible through deployed environment	PENDING — isolated local API exercised; no live URL.
MongoDB connected successfully	PASS, TEST ENVIRONMENT — replica-set connections and operations succeeded. Production pending.
CRUD operations work in production	PENDING — selected create/read/update/delete behaviors pass isolated tests.
Authentication works in production	PENDING — local login and active-user checks pass.
Passwords hashed and never stored as plaintext	PARTIAL — bcrypt account hashes exist; plaintext reset credentials remain.
Protected routes require authentication	PASS for tested HTTP routes; GAP for standalone WebSocket connections.
Role-based restrictions work	PASS for tested worker/admin/inspector role gates; inspector data-scope gaps remain.
Input validation works	PARTIAL — ID, ownership, page, enum, and duplicate-link checks exist; complete request schemas pending.
Sensitive configuration uses environment variables	IMPLEMENTED for backend settings; production secret manager pending.
Production secrets not committed	NOT CERTIFIED — no secret values reproduced; full Git history and remote secrets were not audited.
HTTPS enabled for deployed application	PENDING — no deployed HTTPS evidence.
Secure error handling implemented	PARTIAL / FAIL — malformed JSON reveals a local development stack.
Appropriate security headers configured	GAP — sampled protective headers absent.
Rate limiting or equivalent implemented	GAP — no configured limiter found.
MongoDB access appropriately restricted	PARTIAL — local loopback binding; production identity, TLS, and network access unverified.
Security tests completed and documented	PARTIAL — 16 regression tests passed; supplemental checks recorded; production, rate-limit, and DB access tests remain.

Items to complete before submission

Supply the instructor’s name and submission date; confirm team details; provide and test real deployment URLs; address the required security gaps; clarify whether the instructor accepts the native MongoDB driver instead of Mongoose; capture the missing deployment figures. A safe configuration example is included in Appendix B because the backend .env.example is currently absent from the working tree.

11. Evidence and Screenshots

Captured from the running local React frontend. These images show implemented UI, not a deployed URL. No personal account credentials were used.

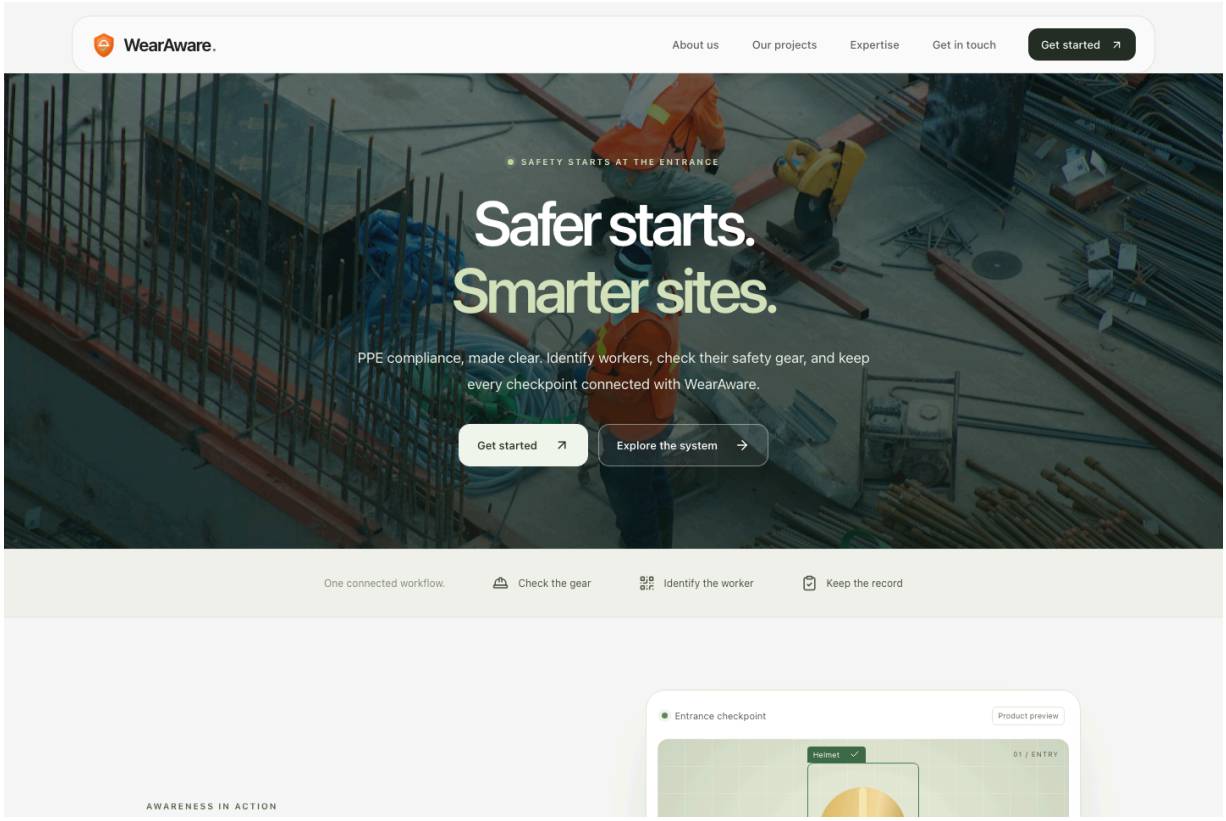


Figure 1. Project Homepage — actual local browser capture, 1440 × 960 viewport.

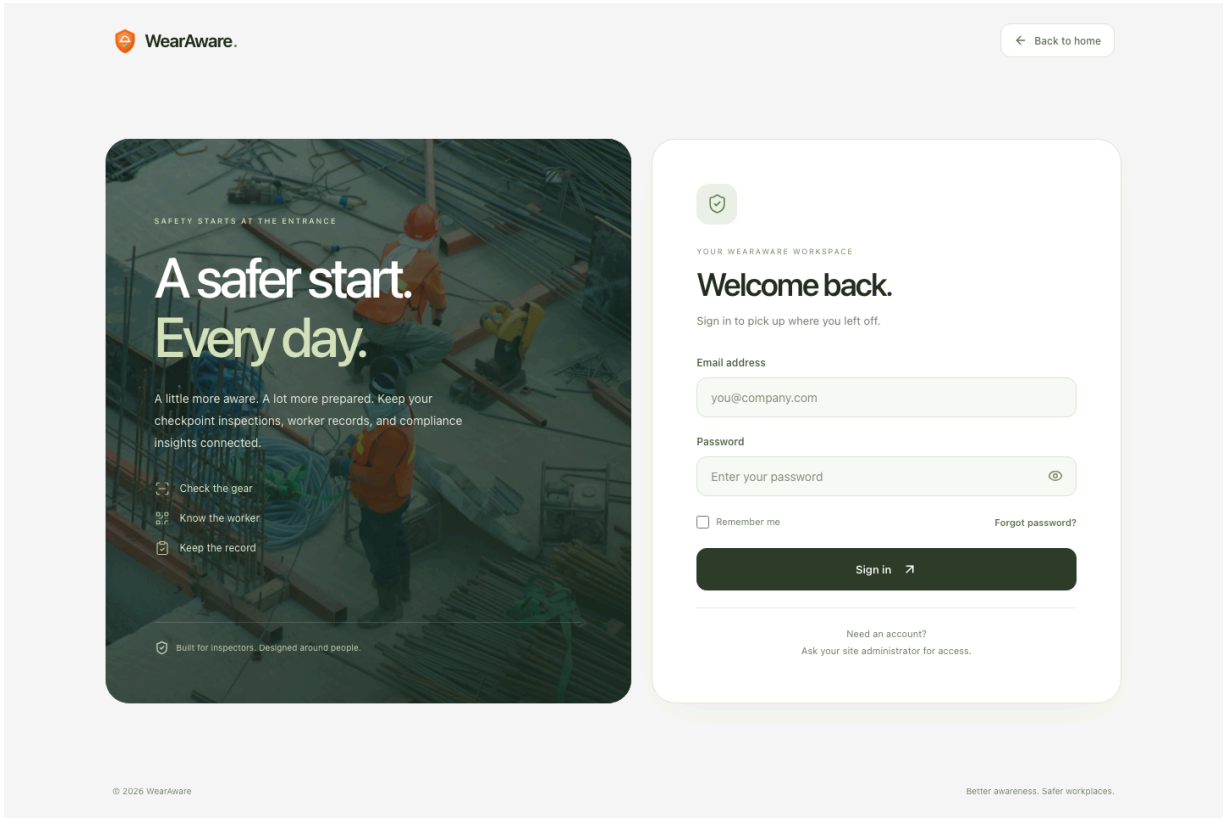


Figure 2. Authentication/Login — actual local browser capture.

11. Evidence and Screenshots (continued)

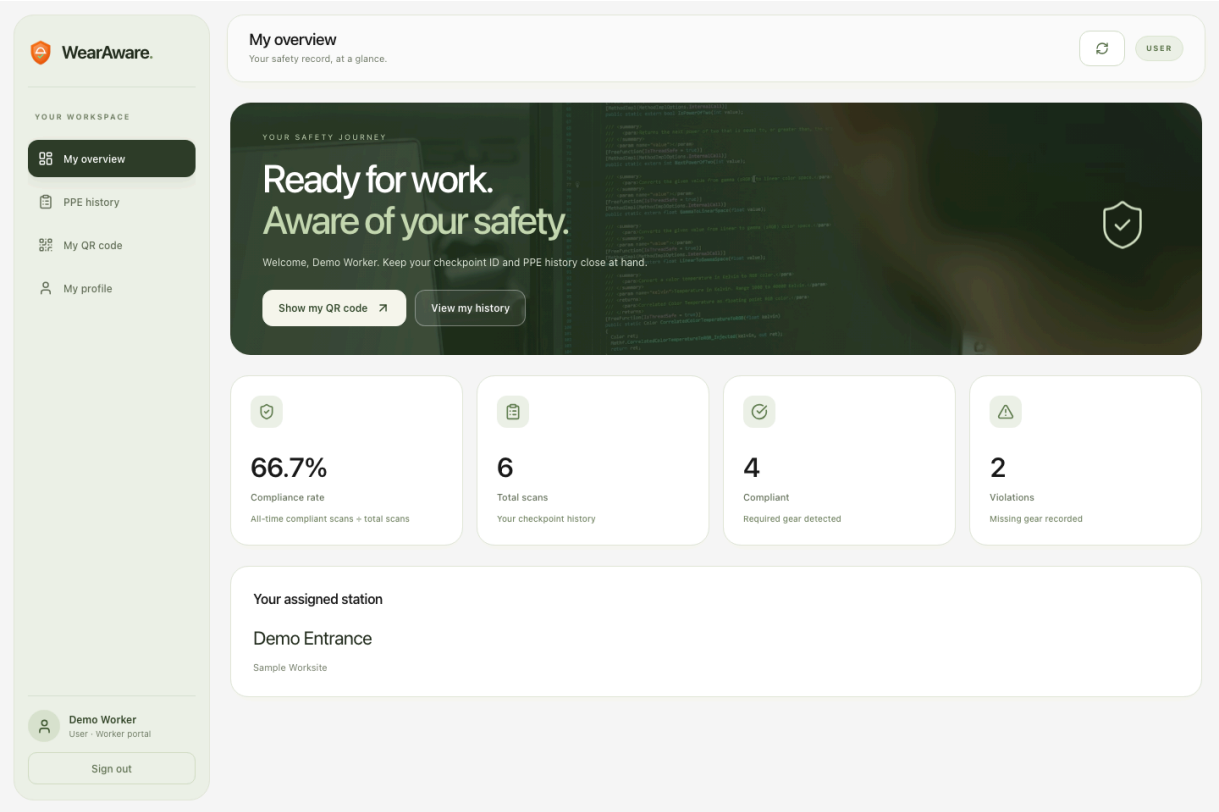


Figure 3. Authorized User Function — actual worker dashboard using synthetic records in an isolated MongoDB test database. Four compliant scans out of six produce 66.7%.

Figure 4 • Recorded HTTP authorization checks (isolated environment)

```
GET /api/users   | authenticated role: user
HTTP 403  {"error":"Access denied."}

GET /api/user/dashboard   | no Authorization header
HTTP 401  {"error":"No token provided."}
```

Figure 4. Restricted/Unauthorized Function — visual transcript of actual HTTP responses, not a browser error-page screenshot.

Figure 5 • Recorded input validation check (isolated environment)

```
GET /api/user/dashboard?page=-1   | authenticated role: user
HTTP 400  {"error":"Invalid page."}
Existing automated tests also reject an object-valued worker_id and duplicate worker links.
```

Figure 5. Input Validation — visual transcript of the actual invalid-pagination response.

Capture method: the frontend ran at localhost:5173 in an isolated browser profile. Its API requests were forwarded to a separate test instance of the existing Express app on 127.0.0.1:5011. The displayed worker and station are invented test fixtures; the screenshot is not evidence of production records. The screenshot process did not alter application source or real accounts.

11. Evidence and Screenshots (continued)

Figure 6 • Actual MongoDB query output (synthetic test records)

```
Collection: workers
id: 11 | employee_id: WA-DEMO-001 | full_name: Demo Worker
device_id: 1 | position: Construction Worker | status: active

Collection: detections
id: 1 | worker_id: 11 | device_id: 1 | result: compliant
detected_ppe: [helmet, vest] | missing_ppe: []

users.worker_id index: unique = true
partialFilterExpression: {worker_id: {$type: "number"}}
Source: security-probes.json; no real worker information or credentials shown.
```

Figure 6. MongoDB Data — recorded query/index output from the disposable test database, formatted for readability. This is not a Compass screenshot.

Requested Figure	Evidence Status
Figure 7. Deployed Frontend	PENDING — capture the publicly hosted frontend with its actual HTTPS URL after deployment.
Figure 8. Deployed Backend/API	PENDING — capture a successful request to the deployed API without exposing authorization tokens.
Figure 9. HTTPS	PENDING — capture certificate/connection evidence for the actual frontend and API domains.

Figure 10 • Actual test summary and supplemental security checks

```
npm --prefix src/wearawarebackend test
16 tests | 16 passed | 0 failed | 0 skipped
Worker ownership, role restrictions, links, migration and repository checks passed.

Supplemental HTTP probes:
Wrong password: 401 | Missing token: 401 | Wrong role: 403
Invalid page: 400 | Own dashboard: 200 | Compliance: 66.7%
Malformed JSON: 400 HTML response containing a stack and local file path.
Security headers sampled: CSP absent; HSTS absent; X-Content-Type-Options absent.
Local and isolated evidence only; production security remains unverified.
```

Figure 10. Security Testing Result — recorded test summary and supplemental probe outcomes. Full logs accompany this document.

Reflection Questions

What changed when your application moved from local development to deployment?

A completed production move has not yet been verified, so we cannot report it as finished. Our local build depends on localhost frontend, API, inference, and WebSocket endpoints. Deployment will require configurable service URLs, trusted HTTPS/WSS, production CORS and secrets, a restricted database connection, and a verified migration/initialization process. Running Compass alone does not start the frontend or Express API.

What security weakness did your testing reveal?

A malformed JSON request produced an HTML response containing a development stack trace and filesystem path. Source review also found plaintext temporary passwords in the reset-request collection, unauthenticated WebSocket broadcasts, missing request throttling/security headers, and inspector data-scope gaps. Passing authentication tests did not prove that all access paths were safe.

How did your group address the identified weakness?

The current worker portal already applies server-derived ownership, current-database role checks, unique worker-account links, and a read-only dashboard. Automated tests verify those protections, including forged IDs and stale role claims. The newly documented error, recovery, WebSocket, and inspector-scope findings remain unresolved in the reviewed code. They are listed as required follow-up work rather than described as completed fixes.

What additional security improvement would you implement with more development time?

We would replace plaintext reset passwords with expiring single-use reset tokens, authenticate and scope WebSocket subscriptions, enforce inspector ownership on every relevant route, centralize validation and safe errors, and add throttling plus deployment-appropriate headers. We would then verify TLS, least-privilege database access, backup recovery, and the same role/data-ownership cases on the actual deployed services.

12. Final Project Information

Required Item	Submission / Link
Source Code Repository	https://github.com/voncheems/WearAwareRM — report includes local changes not necessarily pushed.
Live Frontend / Backend	Not supplied or verified; local frontend :5173 and API :5000 are not public deployment links.
Database Configuration Evidence	Sections 7–8; Figure 6; evidence/security-probes.json; local launcher source [S11].
.env.example	Safe replacement example included as backend.env.example in this deliverable and Appendix B; backend working-tree example is absent.
Architecture Diagram / RBAC	Architectures A and B in Sections 2 and 8.1; RBAC matrix in Section 5.1.
Security Testing Results	Section 9 and Figure 10; evidence/backend-tests.txt; evidence/security-probes.json.
Documentation	WearAware_Checkpoint_02.docx and accompanying evidence folder.
Demo Video (if required)	Not supplied; record the final deployed workflows if required by the instructor.

13. Suggested Evaluation Rubric

Retained from the supplied checkpoint template. Points are available marks, not self-awarded scores.

Criterion	Points
MERN Architecture & Integration	15
Project Functionality / CRUD	15
Authentication & Password Security	10
Authorization & RBAC	10
Input Validation & Secure API	10
Database Security & Least Privilege	10
Deployment & HTTPS	15
Security Testing & Evidence	10
Documentation & Presentation	5
TOTAL	100

Appendix A. Source Evidence Index

Paths are relative to the wearaware application directory in the WearAwareRM repository. References describe the local files reviewed on 23 September 2026.

ID	Source	Used For
S1	src/App.jsx; src/components/AboutUs.jsx	Role-based UI and team metadata.
S2	package.json; src/wearawarebackend/package.json	Actual frontend/backend dependencies and test commands.
S3	src/wearawarebackend/server.js	Login (137), account creation (210), scan creation (324), reset workflow (446–532), startup/WebSocket (542 onward).
S4	src/wearawarebackend/middleware.js	JWT verification, active-account and role checks.
S5	src/wearawarebackend/database.js; repository.js	Native driver, indexes, references, normalization, transactions, statistics.
S6	src/wearawarebackend/routes/user.js; worker-access.js	Worker ownership, pagination, and account-link validation.
S7	src/wearawarebackend/test/*.test.js	16 executed regression tests and assertions.
S8	src/wearawarebackend/routes/workers.js; devices.js; admin.js	Resource CRUD permissions and broad registry reads.
S9	src/wearawarebackend/Inspector.js	Inspector scoping, notifications, assignments, overrides.
S10	src/components/UserDashboard.jsx; LoginPage.jsx; PPEDetectionTab.jsx	Worker UI, token storage, QR and PPE service integration.
S11	src/wearawarebackend/scripts/start-local-mongo.js; scripts/migrate-postgres.js	Local binding/replica set and migration prerequisites.

Appendix B. Safe Configuration Example

This example contains placeholders only. It is included as backend.env.example alongside the report, without restoring or changing the application’s configuration. Do not commit real credentials. Frontend URLs are currently hardcoded and require a separate code change before hosting.

```
# Local development example only — no real credentials.

PORT=5000

WS_PORT=8080

MONGODB_URI=mongodb://127.0.0.1:27017/wearaware?replicaSet=wearaware-rs

MONGODB_DB=wearaware

JWT_SECRET=replace-with-a-long-random-secret

EMAIL_USER=

EMAIL_PASS=

# Legacy PostgreSQL settings: needed only for migration/rollback.

DB_HOST=localhost

DB_PORT=5432

DB_NAME=wearaware

DB_USER=replace-with-migration-user

DB_PASSWORD=replace-with-migration-password
```

Local run and verification notes

The root frontend package uses npm run dev; it has no npm start script. The backend uses npm --prefix src/wearawarebackend start. The local database helper is npm --prefix src/wearawarebackend run db:local. Start only the components needed for the intended environment, and ensure the required database migration state and secrets are configured. Compass is optional for viewing data and is not the process that runs the application.

To repeat the existing regression suite: npm --prefix src/wearawarebackend test. It creates temporary replica-set databases; it does not test the production database. The supplemental JSON file records actual isolated observations from this report preparation. Synthetic fixtures are intentionally separate from real worker data.

Files included with this activity

- WearAware_Checkpoint_02.docx — editable report following all 13 template sections.
- evidence/ — browser captures, architecture diagrams, formatted evidence panels, raw test log, and supplemental JSON.
- backend.env.example — safe configuration example using placeholders.
- README.md — remaining information and submission notes.

Before printing, fill in the instructor and intended submission date, confirm group membership, and replace pending production entries only after those steps are actually completed. Review the print preview for any device-specific font or pagination changes.

WEARAWARE CHECKPOINT 02 - IMPLEMENTATION UPDATE

24 September 2026

This update records functionality, security, and deployment work implemented after the original Checkpoint 02 report. It is appended to the report so the submission reflects the current demonstration build.

DEPLOYED DEMO

The React/Vite frontend is deployed at https://wearaware-demo.vercel.app over HTTPS. For the demonstration, the Express API, MongoDB database, and FastAPI/YOLO PPE service remain on the presentation laptop. The Vercel frontend reaches the API through a Cloudflare Quick Tunnel. The laptop, local MongoDB, Express API, Python model service, and tunnel must remain running during a demonstration. This is a temporary demo architecture, not a permanent production backend; a new tunnel URL requires a frontend redeploy.

RECENT FUNCTIONALITY

- Added the User role, linked to a worker account. Users can view their own QR code, inspection history, and personal compliance rate. Server-side ownership checks prevent access to another worker records.
- Completed admin user management: role selection, worker linking, account active status, and unique worker association for User accounts.
- Updated admin and inspector interfaces with the common responsive design and working logout controls.
- Improved QR lookup resilience with a transient-failure retry and demo-appropriate timeouts. Completed scans return to QR mode for the next worker.
- Detection lists now send a has_photo flag instead of every base64 camera image. The image loads only after an inspector or administrator opens the record, reducing initial dashboard payload.
- Improved PPE inference: when YOLO identifies a person but neither a helmet nor vest, the service creates inferred no-helmet and no-vest violations. Camera input confidence was reduced from 0.35 to 0.25 while class-specific PPE thresholds remain active.

SECURITY CONTROLS IMPLEMENTED

JWT authentication with active-account and current-database role checks protects Administrator, Inspector, and User routes. Server-side role authorization and inspector station/worker scoping restrict records to the assigned account. The API uses typed request validation, MongoDB collection validators, unique indexes, safe error responses, constrained image validation, Helmet headers, explicit CORS origins, request-size limits, authentication rate limits, and password-strength checks.

Password recovery uses single-use, expiring links and stores a hash rather than a reusable plaintext password. WebSocket alerts authenticate before use and are scoped to the assigned inspector. AI uploads require an authenticated inspector, accept only bounded JPEG/PNG input, retain the service key only on the backend, and return safe failures. Browser bearer tokens currently use localStorage; an HttpOnly-cookie session with CSRF protection remains future work.

VALIDATION

The frontend production build completed successfully before deployment. ESLint reported no errors; one pre-existing InspectorDashboard React Hook dependency warning remains. The backend suite passed 27 of 27 tests, covering authorization, validation, rate limiting, recovery, WebSocket scope, AI forwarding, database validation, and User-role ownership. The FastAPI health endpoint authenticated successfully with the backend-held key, and the Python security suite passed with a fake predictor. These AI checks validate request handling and security behavior, not real-world model accuracy.

CURRENT REQUEST FLOW

Vercel React frontend -> Cloudflare Quick Tunnel (HTTPS) -> Express API on the demo laptop -> local MongoDB. PPE scans additionally flow from Express to the localhost FastAPI/YOLO service using a private service key. Database credentials, JWT secrets, and the AI key are not included in the frontend build.

WEARAWARE - EASY-TO-VERIFY IMPLEMENTATION PROOF

For evaluator use - 24 September 2026

LIVE DEMO

Open: https://wearaware-demo.vercel.app

The page is publicly accessible over HTTPS. The group can provide the demo accounts during presentation. The backend and AI model run on the demo laptop, so that laptop and its Cloudflare tunnel must be running while it is reviewed.

WHAT TO OPEN	WHAT YOU WILL SEE	WHAT IT PROVES
Administrator dashboard	Sidebar, logout, Users, Workers, Stations, and compliance records	Role-specific admin management works.
Users page	A User account linked to a worker profile and active status	Role linking is part of the application.
Inspector PPE Scanner	Scan a worker QR, allow camera, then show helmet/vest or remove PPE for a violation result.	QR lookup, camera scanning, AI analysis, and saving results work.
Inspector proof photos	Detection records first show a camera icon. Click it to load.	Photos load only when opened, improving speed.
User dashboard	Personal QR, own history, and compliance rate.	A User can see only their own worker data.

SECURITY PROOF IN PLAIN LANGUAGE

1. Sign in as each role. Each account opens a different dashboard.
2. Try to reach an admin or inspector page using a User account. It is blocked.
3. Open a User profile. The records belong only to the linked worker.
4. In the scanner, camera images go through the protected backend; the browser does not receive the private AI service key.
5. Use Forgot Password. The reset link expires and is usable only once.

TEST EVIDENCE

The backend automated regression suite passed 27 of 27 tests. It covers login, roles, blocked unauthorized routes, request validation, password recovery, rate limiting, inspector scope, WebSocket alert scope, AI upload protection, and User-to-worker ownership. The real model service health check also passed.

IMPORTANT DEMO LIMITATION

Vercel hosts the frontend. The API, database, and AI model are intentionally kept on the group laptop for this free demonstration. This proves the complete flow during a live demo, but it is not a permanent hosted backend.